

Atividade de análise de dados “Complete a Lógica”.

Contexto da atividade

Em uma empresa de desenvolvimento de software, os programadores recebem frequentemente códigos iniciados por outros membros da equipe. Nem sempre o sistema está completo, e cabe ao desenvolvedor analisar o problema, compreender a lógica existente e implementar a parte que falta.

Nesta atividade, cada dupla receberá pequenos trechos de código em diferentes linguagens de programação, como Java, JavaScript, Python, PHP ou C#. Os códigos terão partes incompletas relacionadas a estruturas de programação e estruturas de dados.

O objetivo não será apenas “programar”, mas principalmente **entender o problema, identificar a estrutura correta e justificar a solução escolhida.**

SITUAÇÃO-PROBLEMA

O sistema possui uma tela de login simples conectada ao banco de dados `usuarios.sql`.

O chefe solicitou uma melhoria de segurança:

Ao realizar 3 tentativas incorretas de login, o usuário deverá ficar bloqueado por 3 minutos.

SUA MISSÃO (EM DUPLAS)

Analisar o trecho de código ao lado e responder:

1. Qual é o problema apresentado?
2. Que parte do código está faltando?
3. Qual estrutura deve ser utilizada?
4. Como vocês completariam o trecho?
5. Por que essa solução é adequada?

TRECHO DE CÓDIGO INCOMPLETO (exemplo em Python)

```
1 def login(email, senha):
2     usuario = buscar_usuario(email)
3     if not usuario:
4         return "Usuário não encontrado!"
5
6     # VERIFICAR SE USUÁRIO ESTÁ BLOQUEADO
7     # COMPLETE A LÓGICA AQUI
8
9
10
11 if verificar_senha(senha, usuario['senha']):
12     # LOGIN CORRETO: ZERAR TENTATIVAS
13     # COMPLETE A LÓGICA AQUI
14     return "Login realizado com sucesso!"
15 else:
16     # LOGIN INCORRETO: INCREMENTAR TENTATIVAS
17     # COMPLETE A LÓGICA AQUI
18
19     return "E-mail ou senha incorretos!"
```

Funções disponíveis:

- `buscar_usuario(email)`
- `verificar_senha(senha, hash)`
- `atualizar_usuario(id, dados)`
- `agora()` # retorna data/hora atual
- `minutos_decorridos(data_hora)`

DICAS PARA RESOLVER

- ✓ Criar uma variável para contar as tentativas incorretas.
- ✓ Verificar se o usuário já está bloqueado antes de tentar o login.
- ✓ Registrar o horário do bloqueio.
- ✓ Comparar o horário atual com o tempo de bloqueio.
- ✓ Após 3 minutos, liberar nova tentativa.
- ✓ Ao login correto, zerar o contador de tentativas.

FLUXO ESPERADO

SUGESTÃO DE ESTRUTURA DE DADOS (exemplo de tabela no banco de dados)

Tabela: `usuarios`

id (PK)	email	senha	tentativas_erradas	bloqueado_ate
1	joao@email.com	*****	0	NULL
2	maria@email.com	*****	2	NULL
3	pedro@email.com	*****	3	2025-05-20 10:35:00

tentativas_erradas: quantidade de tentativas incorretas consecutivas
bloqueado_ate: data e hora até quando o usuário ficará bloqueado (NULL = não está bloqueado)

COMPORTAMENTO ESPERADO (EXEMPLO)

Cada dupla deverá analisar os trechos de código fornecidos e responder:

1. Qual é o problema apresentado?
2. Que parte do código está faltando?
3. Qual estrutura de programação ou estrutura de dados deve ser utilizada?
4. Como a dupla completaria o trecho?
5. Por que essa solução é adequada?

Exemplo de situação-problema

Uma empresa possui uma tela de login simples conectada ao banco de dados `usuarios.sql`.

O chefe do setor solicitou uma melhoria de segurança:

Ao realizar 3 tentativas incorretas de login, o usuário deverá ficar bloqueado por 3 minutos.

**ATIVIDADE – COMPLETE A LÓGICA**
SISTEMA DE LOGIN

**Trabalho em duplas**
Análise o trecho de código e complete o que falta.

**MISSÃO**
O chefe pediu que o sistema bloqueie o usuário por 3 minutos após 3 tentativas incorretas de login.

**CENÁRIO**
O sistema possui uma tela de login conectada ao banco de dados `usuarios.sql`.

Ao realizar 3 tentativas incorretas de login, o usuário deverá ficar bloqueado por 3 minutos.

**O QUE VOCÊS DEVEM ENTREGAR**

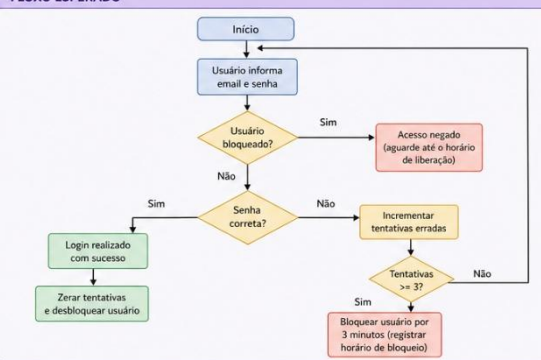
- Qual é o problema apresentado?
- Que parte do código está faltando?
- Qual estrutura deve ser utilizada?
- Como vocês completariam o trecho?
- Por que essa solução é adequada?

**DICAS**

- Criar uma variável para contar as tentativas incorretas.
- Verificar se o usuário já está bloqueado antes de tentar o login.
- Registrar o horário do bloqueio.
- Comparar o horário atual com o tempo de bloqueio.
- Após 3 minutos, liberar nova tentativa.
- Ao login correto, zerar o contador de tentativas.

TRECHO DE CÓDIGO INCOMPLETO (Python)

```
1 import sqlite3
2 from datetime import datetime, timedelta
3
4 def login(email, senha):
5     conn = sqlite3.connect('usuarios.sql')
6     cur = conn.cursor(dictionary=True)
7
8     usuario = cur.execute("SELECT * FROM usuarios WHERE email = ?", (email,)).fetchone()
9     if not usuario:
10         print("Usuário não encontrado!")
11         return
12
13     # ===== COMPLETE A LÓGICA ABAIXO =====
14     # Verificar se o usuário está bloqueado
15     # (Dica: usar campo 'bloqueado_ate')
16     # Se ainda estiver bloqueado, exibir mensagem e retornar
17
18     # Verificar senha
19     # Se incorreta:
20     # - Incrementar 'tentativas_erradas'
21     # - Se chegar a 3, bloquear por 3 minutos (atualizar 'bloqueado_ate')
22     # - Exibir mensagem de erro
23     # Se correta:
24     # - Zerar 'tentativas_erradas' e 'bloqueado_ate' (login realizado com sucesso)
25     # =====
```

FLUXO ESPERADO

SUGESTÃO DE ESTRUTURA DE DADOS (tabela: usuarios)

id (PK)	email	senha	tentativas_erradas	bloqueado_ate
1	joso@email.com	*****	0	NULL
2	maria@email.com	*****	2	NULL
3	pedro@email.com	*****	3	2025-05-20 10:35:00

tentativas_erradas: quantidade de tentativas incorretas consecutivas.
bloqueado_ate: data e hora até quando o usuário ficará bloqueado (NULL = não está bloqueado).

COMPORTAMENTO ESPERADO

A dupla deverá analisar o código entregue e indicar a melhor abordagem para implementar essa regra.

Dicas para os alunos

- Criar uma variável para contar as tentativas incorretas.
- Verificar se o usuário já está bloqueado antes de tentar o login.
- Registrar o horário do bloqueio.
- Comparar o horário atual com o tempo de bloqueio.
- Após 3 minutos, liberar nova tentativa.
- Ao login correto, zerar o contador de tentativas.

Estruturas envolvidas

- Condicional `if/else`;
- Laço de repetição, se necessário;

- Variáveis de controle;
- Registro de data/hora;
- Banco de dados;
- Estrutura de decisão;
- Manipulação de estado do usuário.

Modelo de comando para os alunos

Em duplas, analisem o trecho de código recebido e identifiquem a melhor forma de completar a lógica do sistema.

A resposta da dupla deve conter:

- explicação do problema;
- identificação da estrutura necessária;
- trecho de código ou pseudocódigo com a solução;
- justificativa técnica da escolha;
- possíveis riscos se a regra não for implementada corretamente.

Desafios a serem resolvidos.

Segue link para acesso aos arquivos de cada desafio.

https://drive.google.com/drive/folders/1TDWJz8LjJznR0krCMFICvChoxZcyPs_x?usp=sharing

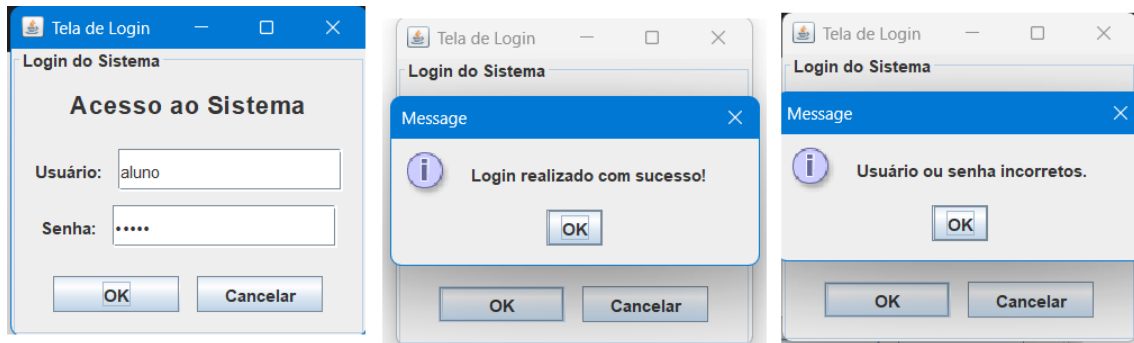
Todos os códigos são funcionais dentro do Netbeans.

Basta apenas importar os arquivos .ZIP e depois realizar a tarefa.

Desafio 1 — Login com bloqueio

Implementar bloqueio após 3 tentativas incorretas.

Estruturas trabalhadas: condicionais, contador, banco de dados, controle de tempo.



Desafio 2 — Carrinho de compras

Um sistema possui uma lista de produtos adicionados ao carrinho. O cliente pode adicionar, remover e alterar a quantidade dos itens.

A dupla deve indicar qual estrutura utilizar para armazenar os produtos e como atualizar o carrinho.

Estruturas trabalhadas: listas, arrays, objetos, dicionários/mapas.

<http://localhost:8080>



Desafio 3 — Fila de atendimento

Uma clínica deseja organizar os pacientes por ordem de chegada. O primeiro paciente a chegar deve ser o primeiro a ser atendido.

A dupla deve completar a lógica de entrada e saída da fila.

Estruturas trabalhadas: fila, FIFO, listas. <http://localhost:9090>



Desafio 4 — Histórico de páginas acessadas

Um navegador precisa permitir que o usuário volte para a página anterior.

A dupla deve indicar qual estrutura representa melhor esse funcionamento.

Estruturas trabalhadas: pilha, LIFO. <http://localhost:9091>



Desafio 5 — Cadastro sem duplicidade

Um sistema de usuários não pode permitir dois cadastros com o mesmo CPF ou e-mail.

A dupla deve propor uma forma eficiente de verificar duplicidade.

Estruturas trabalhadas: conjunto, chave única, busca, banco de dados.



<http://localhost:9095/>

Produto final da dupla

Cada dupla deverá entregar uma ficha técnica contendo:

Nome da dupla:

Linguagem analisada:

Problema identificado:

Estrutura utilizada:

Solução proposta:

Justificativa:

Teste esperado:

Arquivos importantes:

Segue link com os arquivos necessários para instalar as aplicações em Java no netbeans.

- Arquivo zip. Tela de login.
- Arquivo jar sqlite (banco de dados).
- Vídeo para instalação passo-à-passo.

https://drive.google.com/drive/folders/1TDWJz8LjJznR0krCMFICvChoxZcyPs_x?usp=sharing

Orientação para os alunos — Como criar prompts corretos para resolver a atividade

Durante esta atividade, vocês poderão utilizar IA como apoio para análise e raciocínio lógico. Porém, a qualidade da resposta depende diretamente da qualidade do comando enviado.

Um comando mal feito gera respostas ruins.

Um comando bem feito gera respostas úteis.

Por isso, aprender a montar um bom prompt é parte da atividade.

O que é um Prompt?

Prompt é o comando, pergunta ou instrução que você envia para a IA.

Exemplo ruim:

`me ajuda nesse código`

Exemplo bom:

`Tenho uma tela de login em Java com banco SQLite.
Preciso implementar um bloqueio de 3 minutos após 3 tentativas`

incorretas.

Como posso fazer isso usando contador de tentativas e registro de tempo?

Esse comando é claro e objetivo.

A IA entende melhor e responde melhor.

Estrutura de um bom Prompt

Todo bom prompt deve conter:

1. Contexto

Explique o que está acontecendo.

Exemplo:

Estou criando uma tela de login em Java no NetBeans com banco de dados SQLite.

2. Problema

Explique exatamente o que precisa ser resolvido.

Exemplo:

Preciso bloquear o usuário após 3 tentativas incorretas.

3. Objetivo

Diga o que você quer receber.

Exemplo:

Quero uma explicação e um exemplo de código comentado.

4. Modelo simples de Prompt

Estou desenvolvendo _____

O problema é _____

Preciso que a IA me ajude com _____

Quero que a resposta seja _____

Exemplos práticos para esta atividade

Exemplo 1 — Login com bloqueio

Tenho uma tela de login em Java com SQLite.
O usuário padrão é aluno e senha aluno.
Preciso implementar uma regra onde,
após 3 tentativas incorretas,
o sistema bloqueie o login por 3 minutos.
Quero uma sugestão de lógica usando contador
de tentativas e tempo de bloqueio.

Dicas importantes

Seja específico

Quanto mais detalhes, melhor.

Diga a linguagem usada

Java
Python
JavaScript
C#

Explique o erro

Exemplo:

Está aparecendo:
No suitable driver found

Diga o que já tentou

Isso evita respostas repetidas.